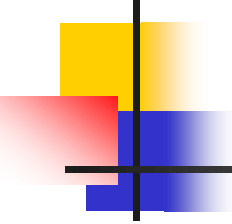


Chapter 13

Instruction-Level Parallelism and Superscalar Processors

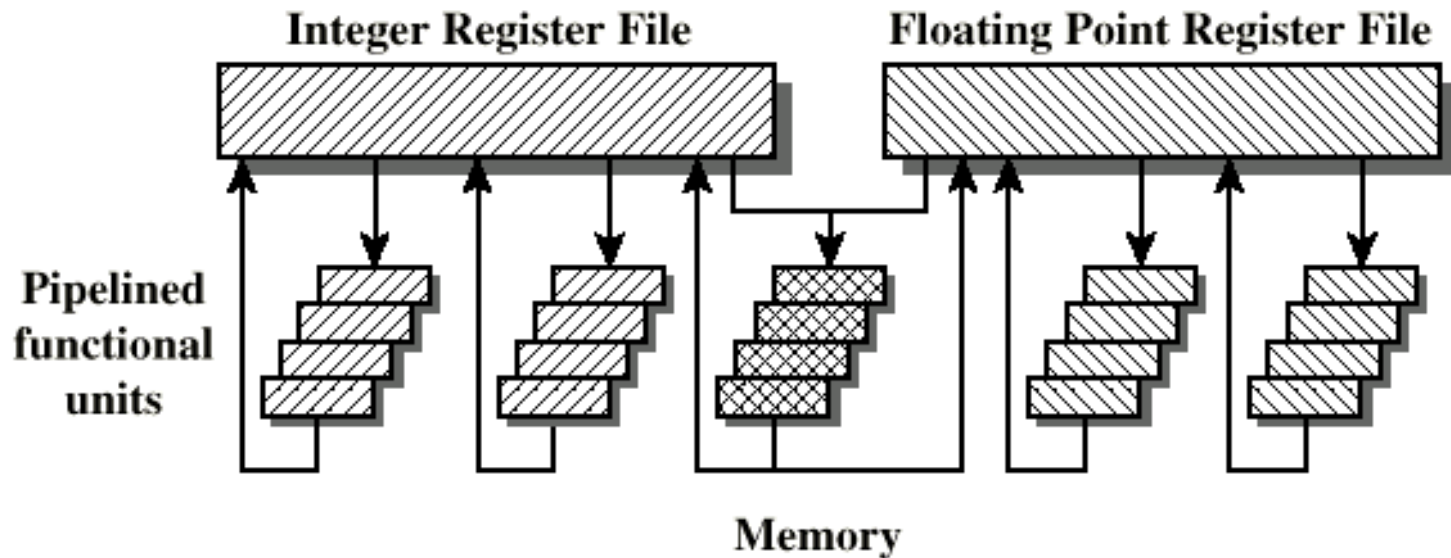


Superscalar Approach

- The superscalar approach has now become the standard method for implementing high-performance microprocessors.
- The term superscalar refers to a machine that is designed to improve the performance of the execution of scalar instructions.
- The essence of the superscalar approach is the ability to execute instructions independently in different pipelines.

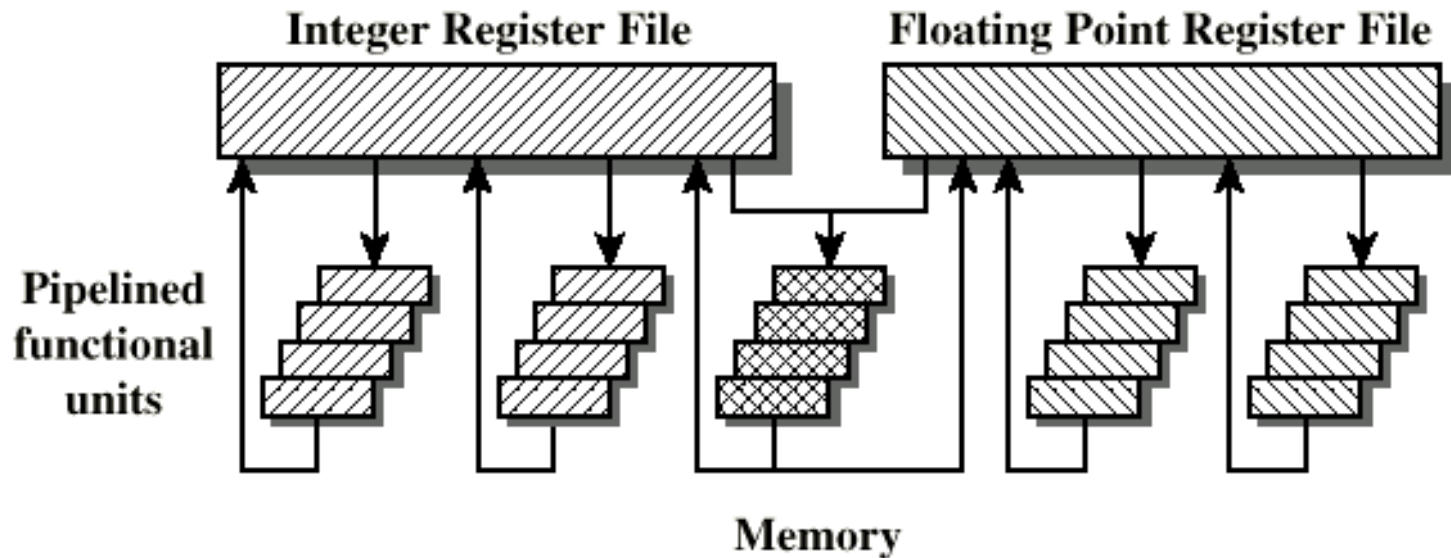
Superscalar Approach

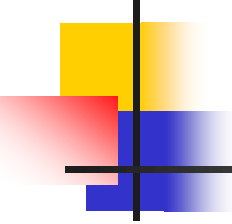
- The concept can be further exploited by allowing instructions to be executed in an order different from the original program order.
- Here, there are multiple functional units, each of which is implemented as a pipeline. Each pipeline supports parallel execution of instructions.



Superscalar Approach

- In this example, the pipelines enable the simultaneous execution of two integer, two floating point, and one memory operation.
- Research indicates that the degree of improvement can vary from 1.8 to 8 times.





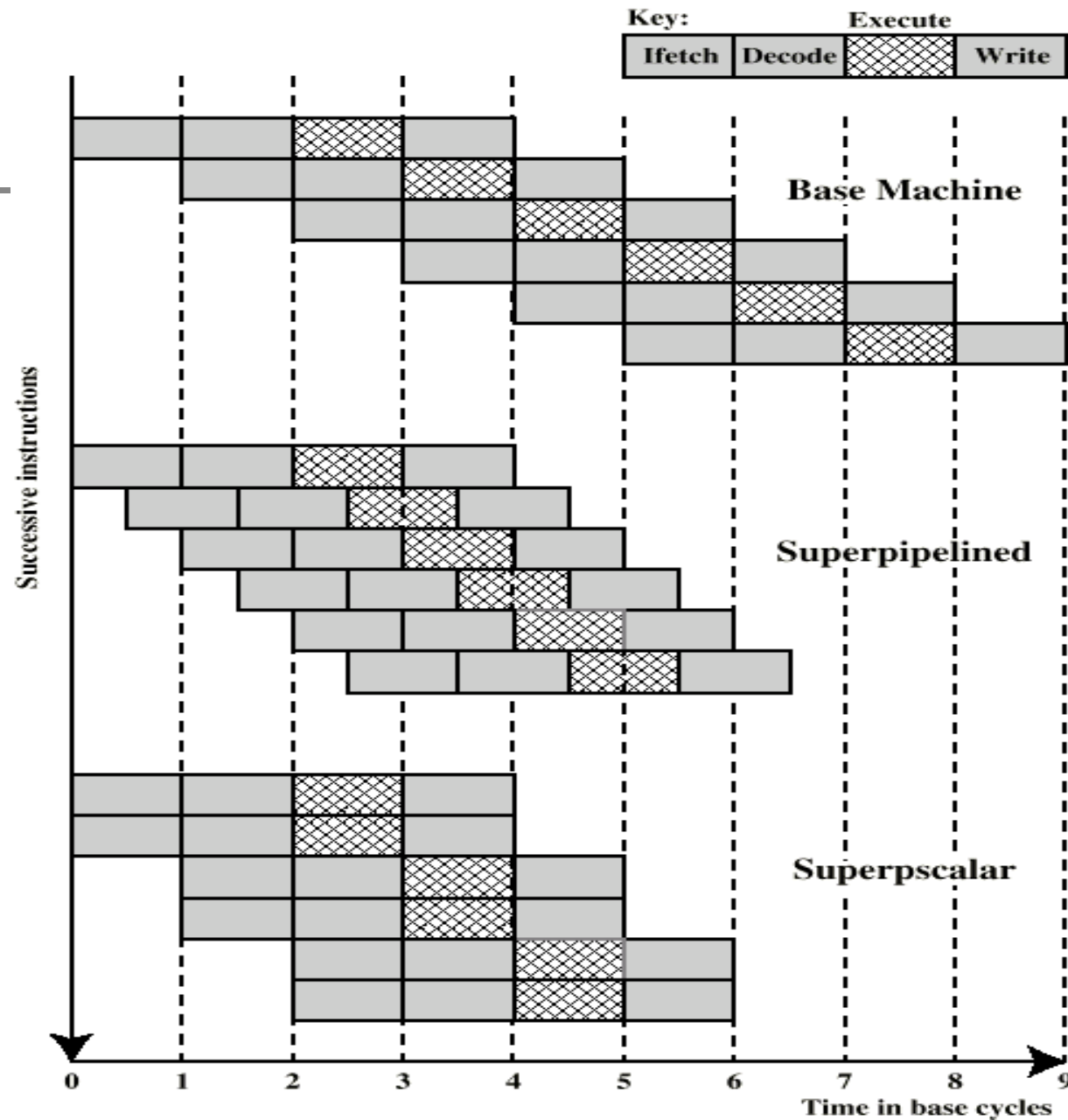
Superscalar vs. Superpipelined

- Superpipelining exploits the fact that many pipeline stages perform tasks that require less than half a clock cycle.
- Thus, a doubled clock cycle allows the performance of two tasks in one external clock cycle (e.g. MIPS R4000).

Superscalar vs. Superpipelined



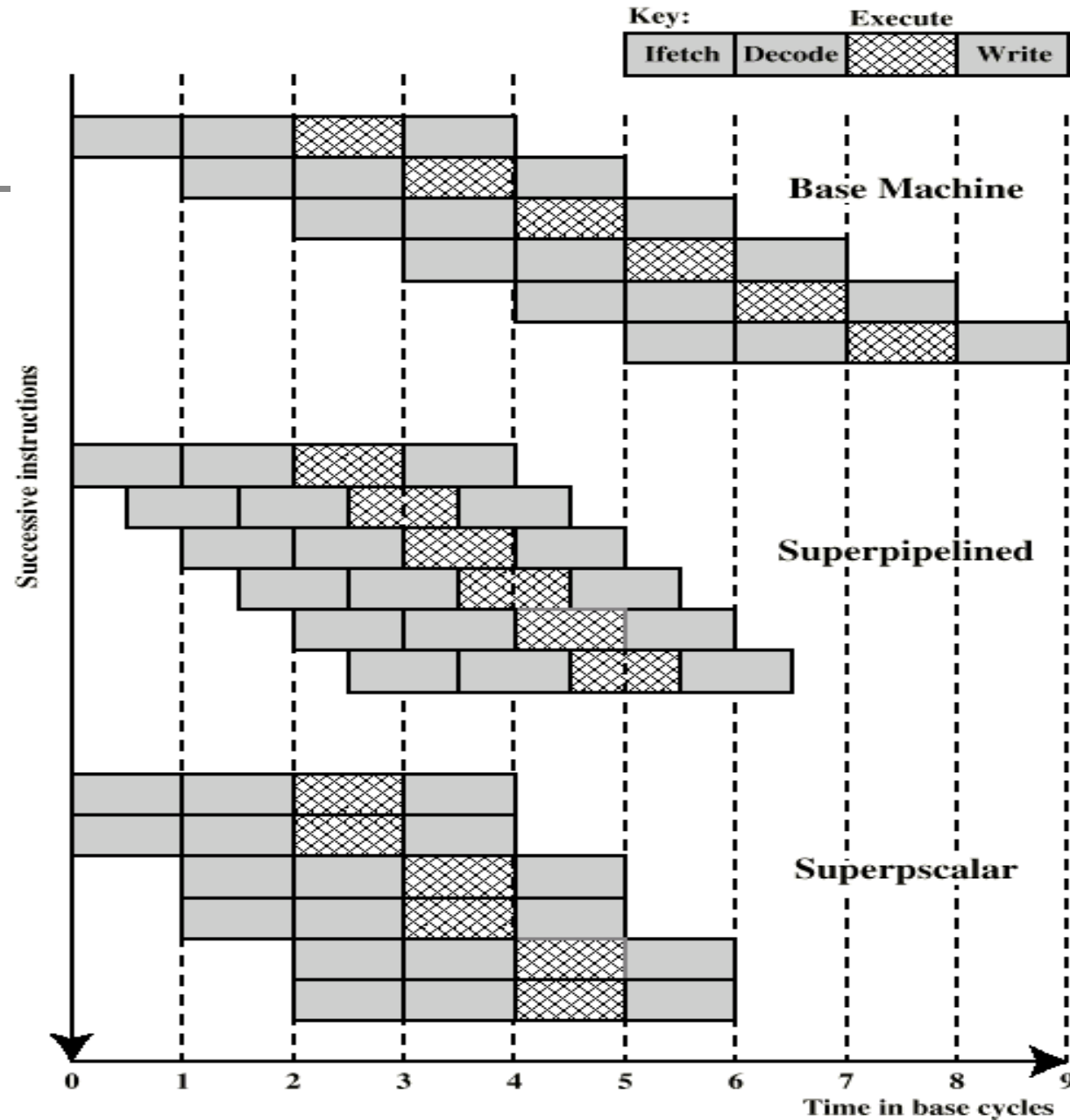
A comparison of a superpipelined and a superscalar approach to a base machine with an ordinary pipeline.



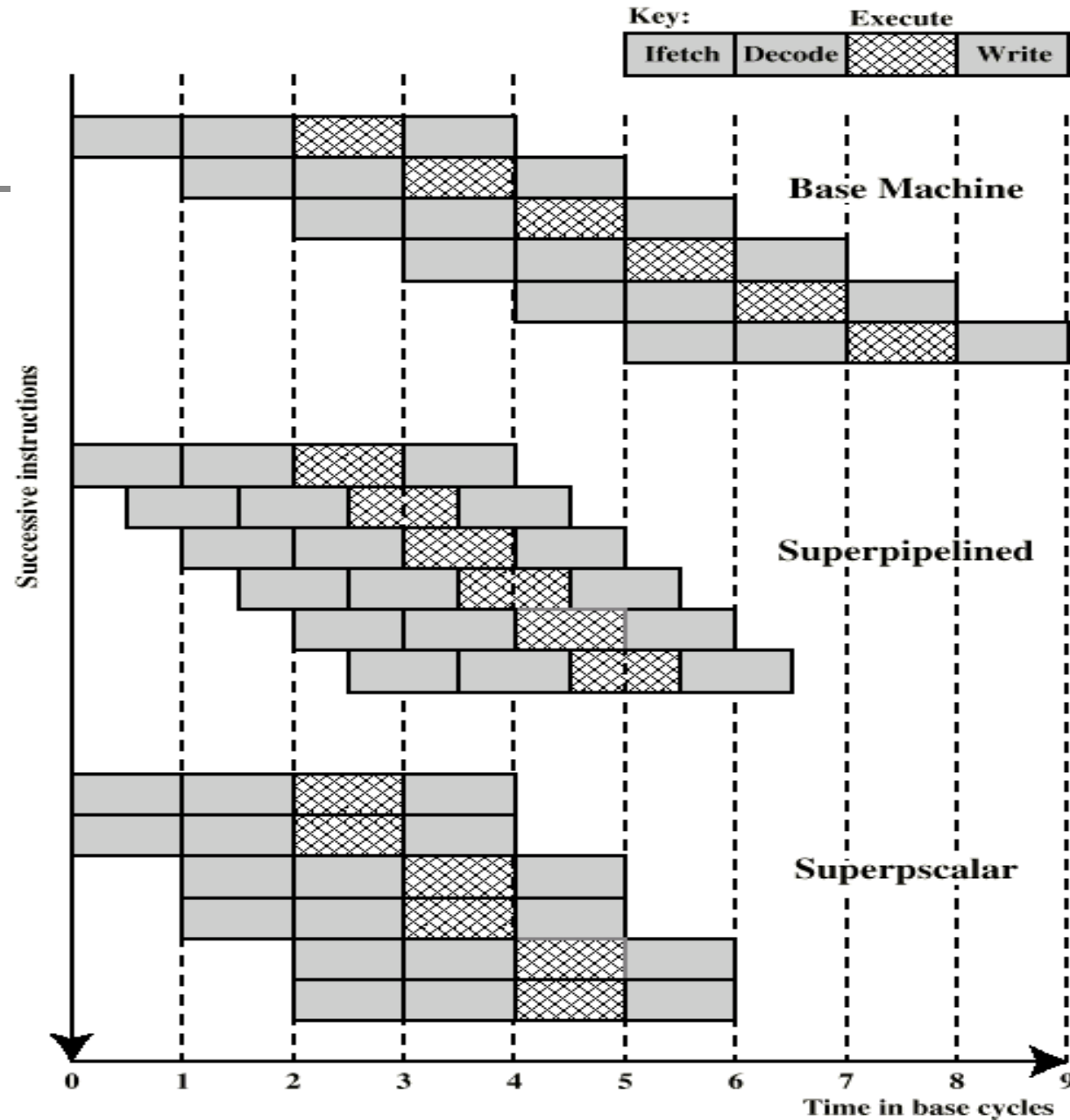
Superscalar vs. Superpipelined



- The pipeline has four stages: instruction fetch, operation decode, operation execution, and result write back.
- The base pipeline issues one instruction per clock cycle and can perform one pipeline stage per clock cycle.
 - Although several instructions are in the pipeline concurrently, only one instruction is in its execution stage at any one time.

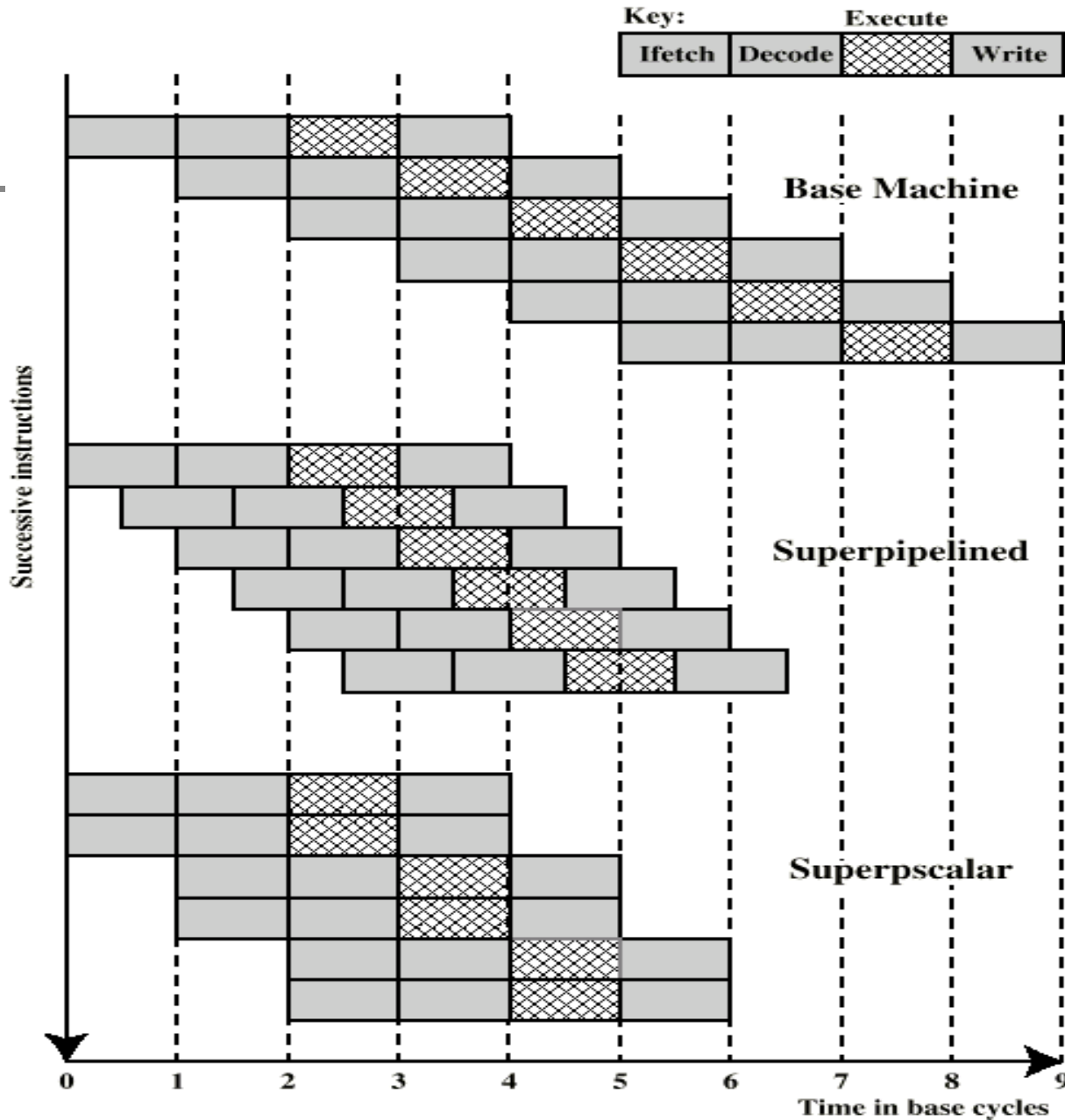


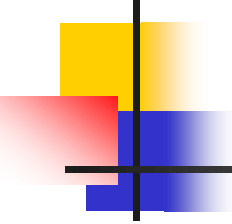
Superscalar vs. Superpipelined



- The superpipelined implementation is capable of performing two pipeline stages per clock cycle (superpipeline of degree 2).
 - i.e. the functions performed in each stage can be split into two nonoverlapping parts which can execute in half a clock cycle.
- The superscalar implementation is capable of executing two instances of each stage in parallel (degree 2).

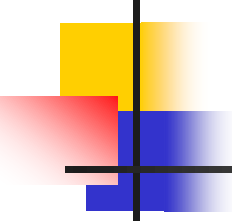
-
- Key:** Ifetch Decode Execute Write
- Base Machine**
- Superpipelined**
- Superscalar**
- Successive instructions
- Time in base cycles





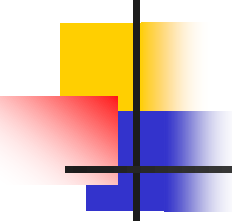
Limitations

- Superscalar approach depends on the ability to execute multiple instructions in parallel.
- **Instruction-level parallelism** refers to the degree to which, on average, the instructions of a program can be executed in parallel.
- A combination of compiler-based optimization and hardware techniques can be used to maximize instruction-level parallelism.



Limitations

- There are five fundamental limitations to parallelism with which the system must cope:
 - True data dependency
 - Procedural dependency
 - Resource conflicts
 - Output dependency
 - Antidependency



True Data Dependency

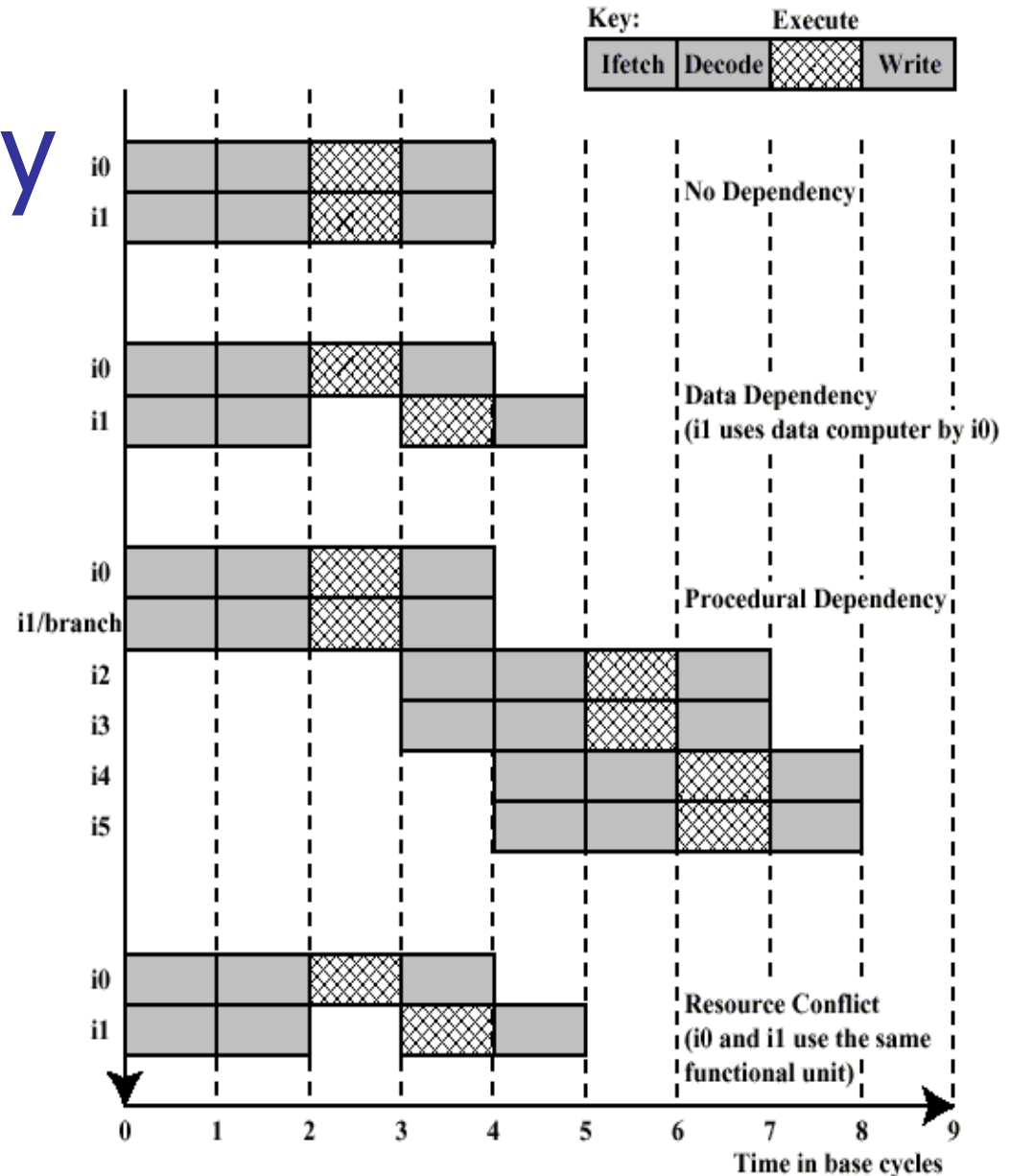
- Consider the following sequence:

```
add    r1, r2    ;load register r1 with the contents of r2
                    ;plus the contents of r1
move   r3, r1    ;load register r3 with the contents of r1
```

- The second instruction can be fetched and decoded but cannot execute until the first instruction executes, as it needs data produced by the first.

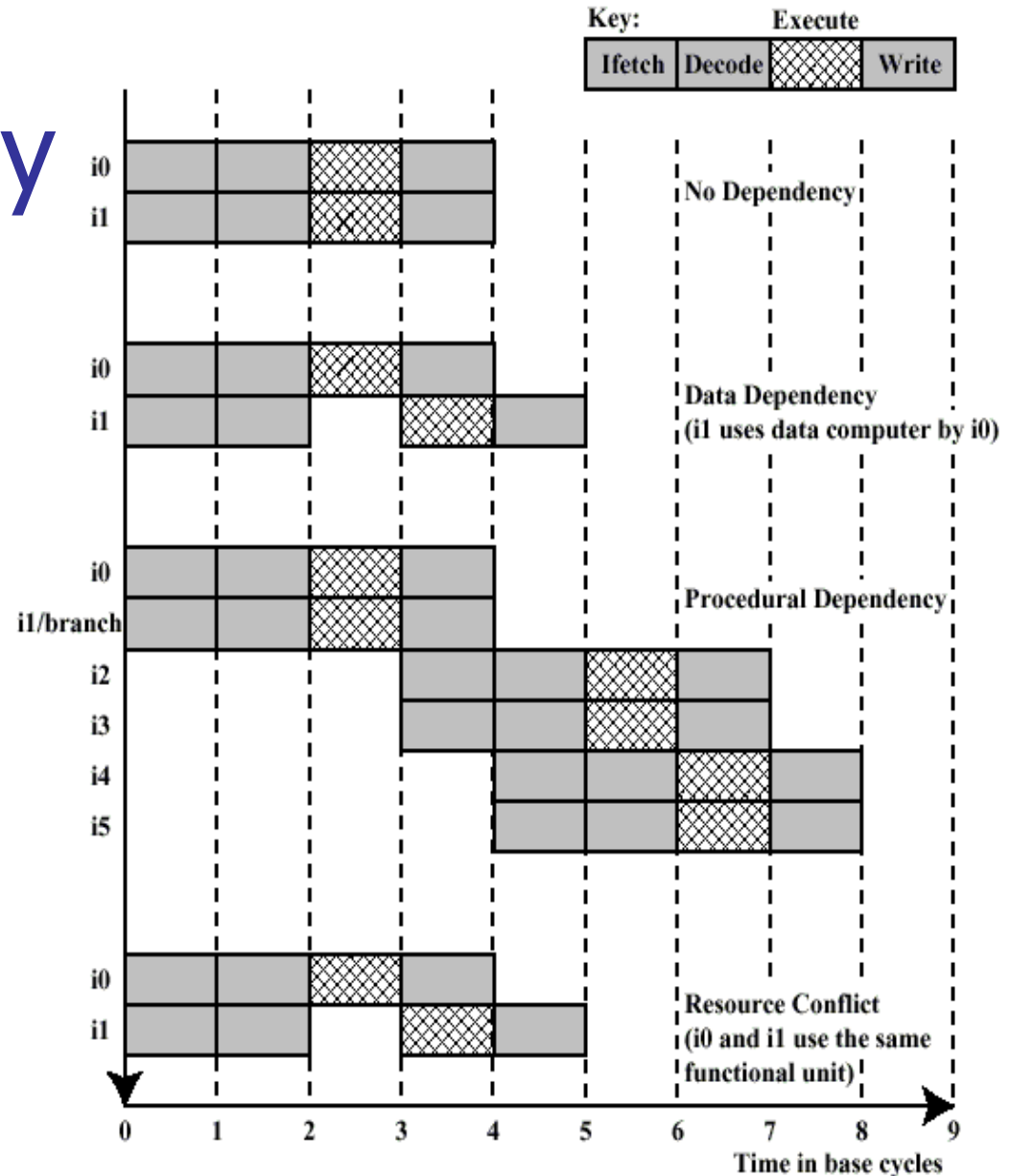
True Data Dependency

- Figure 13.3 illustrates this dependency in a superscalar machine of degree 2.
- With no dependency, two instructions can be fetched and executed in parallel.
- If there is a data dependency between the first and second instructions, then the second instruction is delayed as many clock cycles as is required to remove the dependency.
- In general, any instruction must be delayed until all of its input values have been produced.



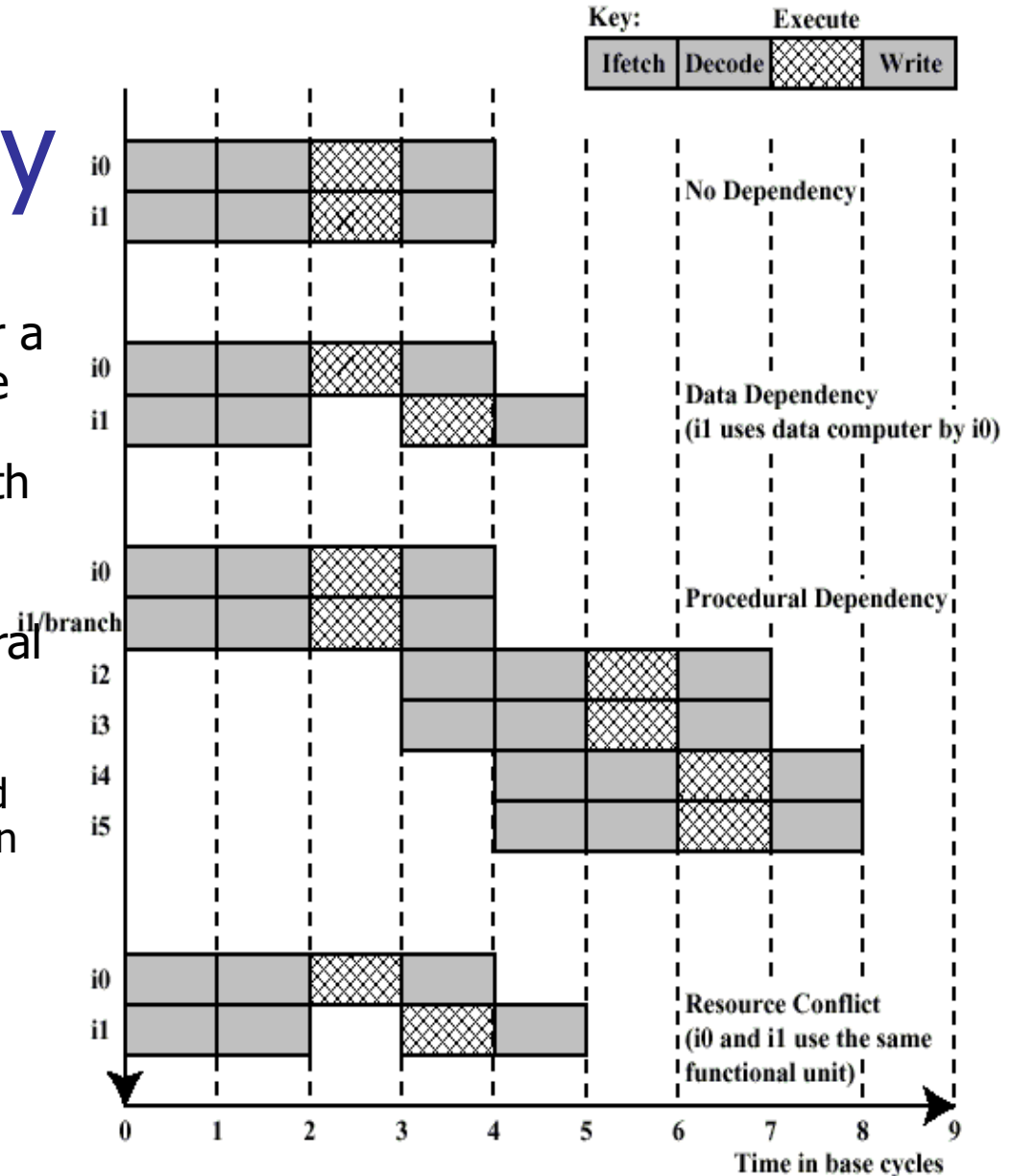
Procedural Dependency

- The presence of branches in an instruction sequence complicates the pipeline operation.
- The instructions following a branch have a procedural dependency on the branch and cannot be executed until the branch is executed.
- Figure 13.3 illustrates the effect of a branch on a superscalar pipeline of degree 2.



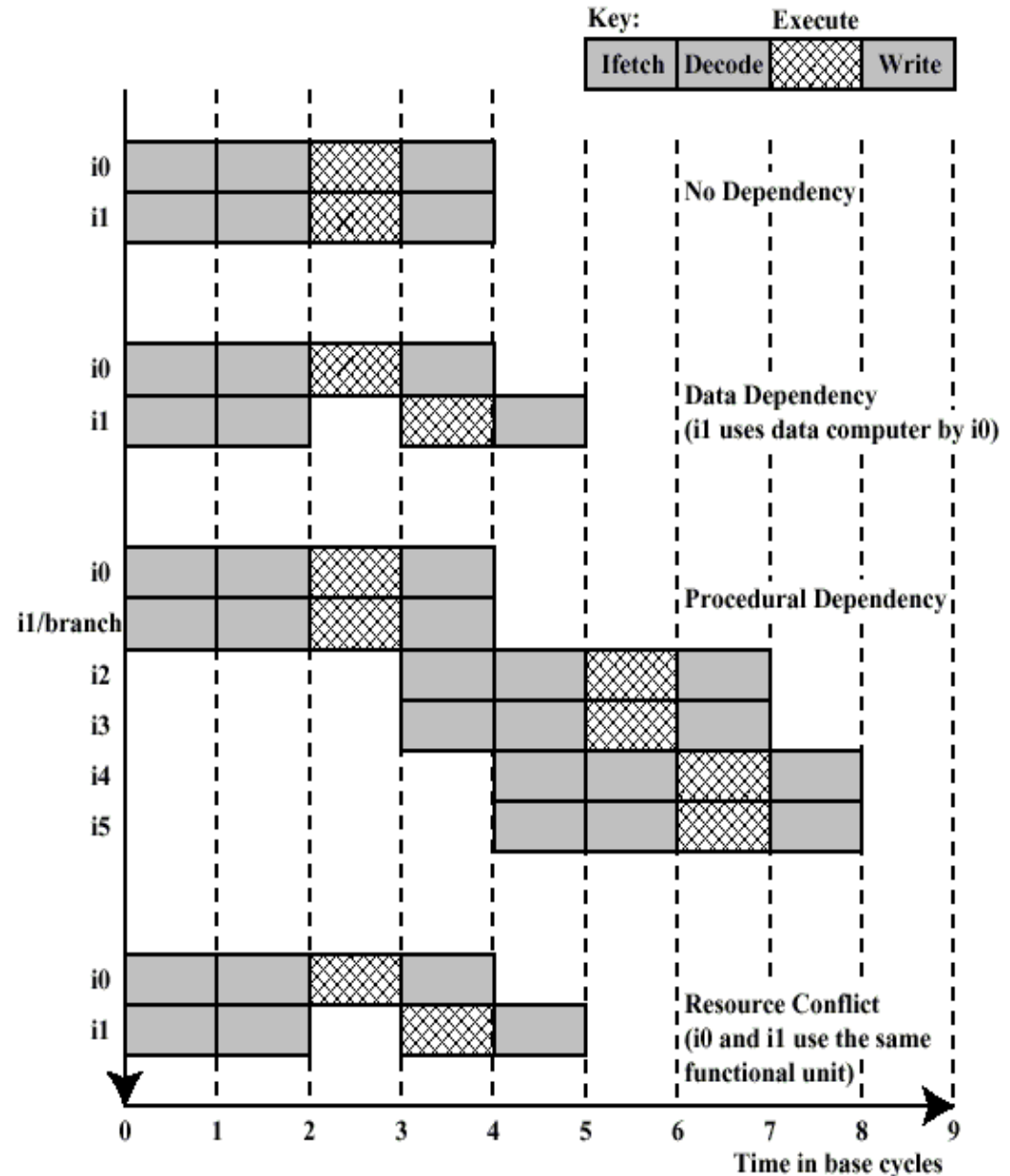
Procedural Dependency

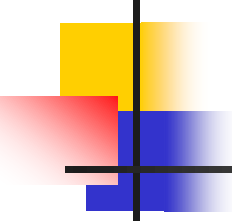
- This dependency is more severe for a superscalar processor than a simple scalar pipeline, as a greater magnitude of opportunity is lost with each delay.
- If variable-length instructions are used, then another sort of procedural dependency arises.
 - Because instruction length is not known, it must be partially decoded before the following instructions can be fetched.
 - This prevents the simultaneous fetching required in a superscalar pipeline.
 - This is one of the reasons that superscalar techniques are more readily applicable to a RISC architecture, with its fixed length.



Resource Conflict

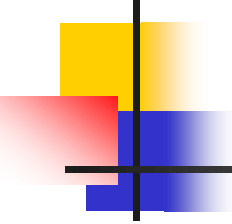
- A resource conflict is a competition for the same resource at the same time. Resources may include memories, caches, buses, register-file ports, and functional units
 - e.g. ALU, adder.
- In terms of the pipeline, a resource conflict exhibits behaviour similar to a data dependency.
- The difference is that conflicts may be overcome by duplication of resources.





Superscalar Limitations

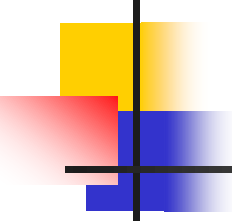
- Output dependencies and Antidependencies will be addressed in the next section.



13.2 Design Issues

Instruction-Level Parallelism and Machine Parallelism

- It is important to distinguish between these two types of parallelism.
 - **Instruction-level parallelism** exists when instructions in a sequence are independent and can thus be executed in parallel by overlapping.



Instruction-Level Parallelism

- For example,

load R1 ← R2

add R3 ← R3, "1"

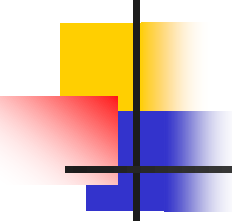
add R4 ← R4, R2

add R3 ← R3, "1"

add R4 ← R3, R2

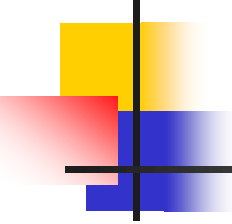
store [R4] ← R0

- The three instructions on the left are independent, and in theory all three could be executed in parallel.
- The three instructions on the right cannot be executed in parallel because the second instruction uses the result of the first, and the third instruction uses the result of the second.



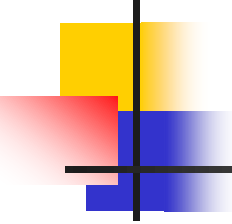
Instruction-Level Parallelism

- Instruction-level parallelism is determined by the frequency of true data dependencies and procedural dependencies in the code.
- These factors are, in turn, dependent on the instruction set architecture and the application.
- Also: operation latency - the time until a result of an instruction is available for use as an operand in a subsequent instruction. How much delay a data or procedural dependency will cause.



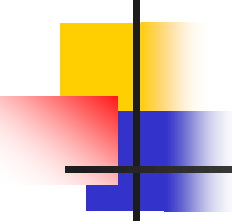
Machine Parallelism

- Machine parallelism is a measure of the ability of the processor to take advantage of the instruction-level parallelism.
- Determined by the number of instructions that can be fetched and executed at the same time (the number of parallel pipelines) and by the speed and sophistication of the mechanisms that the processor uses to find independent instructions.



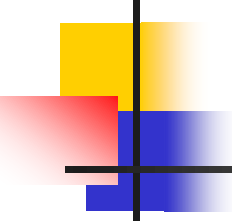
Parallelism

- Both instruction-level and machine parallelism are important factors in enhancing performance.
- A program may not have enough instruction-level parallelism to take advantage of machine parallelism.
- A fixed length instruction architecture (such as RISC), enhances instruction-level parallelism.
- Limited machine parallelism will limit performance no matter what the nature of the program.



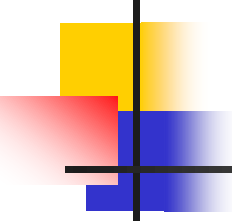
Instruction Issue Policy

- ❑ Processor must be able to identify instruction-level parallelism, and coordinate fetching, decoding and execution of instructions in parallel.
- ❑ Instruction issue: initiating instruction execution in the processor's functional units.
- ❑ Instruction issue policy: the protocol used to issue instructions.
- ❑ The processor is trying to look ahead of the current point of execution to locate instructions that can be brought into the pipeline and executed.



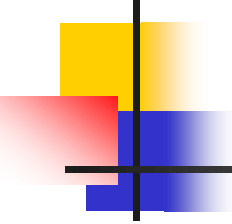
Instruction Issue Policy

- Three types of ordering are important:
 - Order in which instructions are fetched
 - Order in which instructions are executed
 - Order in which instructions update the contents of register and main memory
- The more sophisticated the processor, the less it is bound by a strict relationship between these orderings.



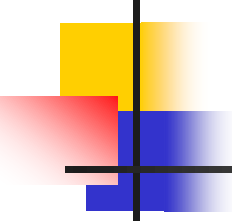
Instruction Issue Policy

- To optimize pipeline utilization, the processor will need to alter one or more of these orderings with respect to the ordering in strict sequential execution.
- The one constraint on the processor is that the result must be correct.
- Dependencies and conflicts must be accommodated.



Instruction Issue Policy

- Instruction issue policies can be grouped into the following categories:
 - In-order issue with in-order completion
 - In-order issue with out-of-order completion
 - Out-of-order issue with out-of-order completion.



In-order issue with in-order completion

- Simplest policy.
- Not even scalar pipelines follow such a simplistic policy.
- It is useful to consider this policy for comparison with more sophisticated policies.

In-order issue with in-order completion

- Superscalar pipeline capable of fetching and decoding two instructions at a time.
- Three separate functional units: integer arithmetic, floating points arithmetic), and two instances of the write-back pipeline stage.

Constraints on the six-instruction code fragment:

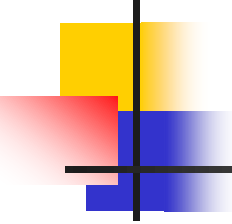
- I1 requires two cycles to execute
- I3 and I4 conflict for the same functional unit.
- I5 depends on the value produced by I4.
- I5 and I6 conflict for a functional unit.

Decode		Execute			Write		Cycle
I1	I2						1
I3	I4	I1	I2				2
I3	I4	I1					3
	I4			I3	I1	I2	4
I5	I6			I4			5
	I6		I5		I3	I4	6
			I6				7
					I5	I6	8

In-order issue with in-order completion

- Instructions are fetched two at a time, and passed to the decode unit.
 - The next two instructions must wait until the pair of decode pipeline stages has cleared.
- To guarantee in-order completion, when there is a conflict for a functional unit, or when a functional unit requires more than one cycle to generate a result, the issuing of instructions temporarily stalls.
- In this example, the elapsed time from decoding the first instruction to writing the last results is eight cycles.

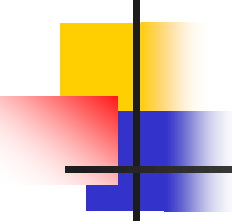
Decode		Execute		Write		Cycle
11	12					1
13	14	11	12			2
13	14	11				3
	14			11	12	4
15	16					5
	16		15	13	14	6
			16			7
				15	16	8



In-order issue with out-of-order completion

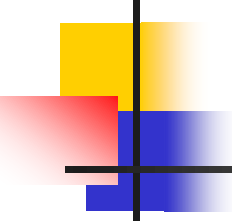
- Out-of-order completion is used in scalar RISC processors to improve the performance of instructions that require multiple cycles.
- Here, I2 is allowed to run to completion prior to I1.
- This allows I3 to be completed earlier, with the net savings of one cycle.

Decode		Execute			Write		Cycle
I1	I2						1
I3	I4	I1	I2				2
	I4	I1		I3	I2		3
I5	I6			I4	I1	I3	4
	I6		I5		I4		5
			I6		I5		6
					I6		7



In-order issue with out-of-order completion

- Any number of instructions may be in the execution stage at any one time, up to the maximum degree of machine parallelism (functional units).
- Instruction issuing is stalled by a
 - resource conflict,
 - data dependency, or
 - procedural dependency.



In-order issue with out-of-order completion

- In addition to the aforementioned dependencies, a new dependency arises: output dependency (or write-write dependency).

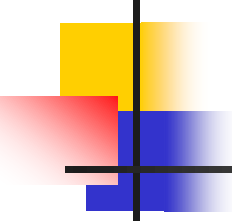
I1: $R3 \leftarrow R3 \text{ op } R5$

I2: $R4 \leftarrow R3 + 1$

I3: $R3 \leftarrow R5 + 1$

I4: $R7 \leftarrow R3 \text{ op } R4$

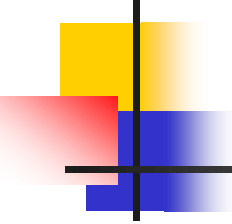
- I2 cannot execute before I1, because it needs the result in register R3 produced in I1 (true data dependency).
- Similarly, I4 must wait for I3.



In-order issue with out-of-order completion

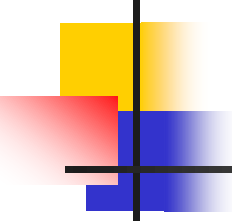
I1: R3 $\leftarrow$ R3 op R5
I2: R4 $\leftarrow$ R3 + 1
I3: R3 $\leftarrow$ R5 + 1
I4: R7 $\leftarrow$ R3 op R4

- What about I1 and I3? – **Output Dependency**
 - There is no true data dependency.
 - However, if I3 completes before I1, then the wrong contents of R3 will be passed to I4 (those produced by I1).
 - I3 must complete after I1 to produce correct output.
 - Issue of third instruction must be stalled.



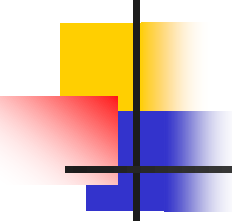
In-order issue with out-of-order completion

- Out-of-order completion requires more complex instruction-issue logic than in-order completion.
- It is more difficult to deal with interrupts (instructions ahead of the interrupt point may have already completed).



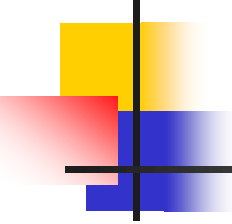
Out-of-order issue with out-of-order completion

- With in-order issue, the processor will decode instructions only up to the point of a dependency or conflict.
- No additional instructions are decoded until the conflict is resolved.
- Thus, the processor cannot look ahead of the point of conflict to subsequent instructions that may be independent of those already in the pipeline.
- To enable out-of-order issue it is necessary to decouple the decode and execute stages of the pipeline.



Out-of-order issue with out-of-order completion

- This is done with a buffer referred to as an instruction window.
- After decoding, the processor places the instruction in the instruction window.
- As long as the buffer is not full, the processor can continue to fetch and decode new instructions.
- When a functional unit becomes available in the execute stage, an instruction from the instruction window may be issued to the execute stage (if it needs that particular functional unit, and no dependencies or conflicts exist).



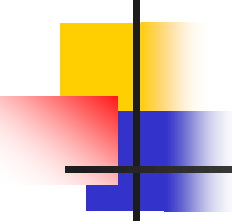
Out-of-order issue with out-of-order completion

- Processor has lookahead capability, and can identify instructions that can be brought into the execute stage.
- Instructions are issued from the instruction window with little regard for their original order.

Out-of-order issue with out-of-order completion

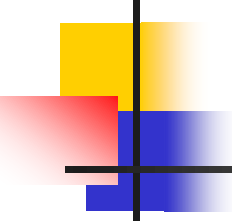
- On each cycle, two instructions are fetched into the decode stage.
- On each cycle, subject to the constraint of the buffer size, two instructions move from the decode stage to the instruction window.
- In this example, it is possible to issue instruction I6 ahead of I5.
 - Recall that I5 depends upon I4, but I6 does not.
- One cycle is saved in both the execute and write-back stages. The end-to-end savings, compared with in-order issue, is one cycle.

Decode		Window	Execute			Write		Cycle
I1	I2							1
I3	I4	<i>I1,I2</i>	I1	I2				2
I5	I6	<i>I3,I4</i>	I1		I3	I2		3
		<i>I4,I5,I6</i>		I6	I4	I1	I3	4
		<i>I5</i>		I5		I4	I6	5
						I5		6



Out-of-order issue with out-of-order completion

- This policy is subject to the same constraints described earlier. An instruction cannot be issued if it violates a dependency or conflict.
- The difference is that more instructions are available for issue, reducing the probability that a pipeline stage will have to stall.

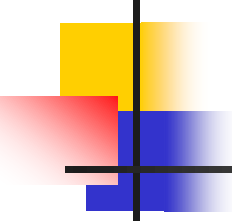


Out-of-order issue with out-of-order completion

- In addition, a new dependency, called an **antidependency**, arises. This is illustrated in the code fragment:

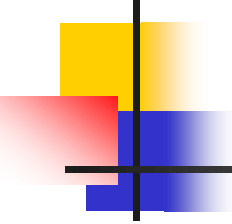
```
I1:    R3 ← R3 op R5
I2:    R4 ← R3 + 1
I3:    R3 ← R5 + 1
I4:    R7 ← R3 op R4
```

- I3 cannot complete execution before I2 begins execution and has fetched its operands.
- This is because I3 updates register R3, which is a source operand for I2.



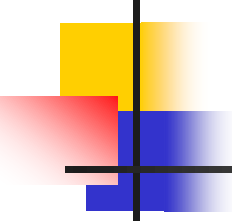
Out-of-order issue with out-of-order completion

- The term **antidependency** is used because the constraint is similar that that of a true data dependency, but reversed: instead of the first instruction producing a value that the second instruction uses, the second instruction destroys a value that the first instruction uses.



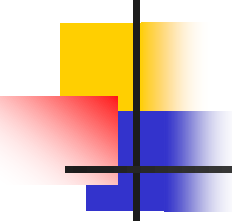
Register Renaming

- When out-of-order instruction issuing and/or out-of-order completion are allowed, this gives rise to the possibility of output dependencies and antidependencies.
 - The values in the registers may no longer reflect the sequence of values dictated by the program flow.
- When instructions are issued / completed in sequence, it is possible to specify the contents of each register at each point in the execution.



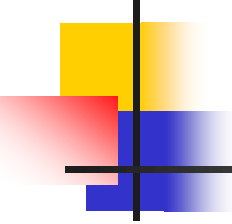
Register Renaming

- With out-of-order techniques, the value of the registers cannot be known just from the dictated sequence of instructions.
- In effect, values are in conflict for the use of registers, and the processor must resolve those conflicts by occasionally stalling the pipeline.
- This problem is exacerbated by register optimization techniques, which attempt to maximize the use of registers, hence maximizing the number of storage conflicts.



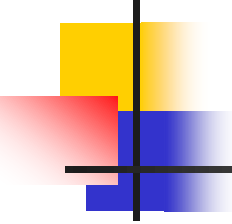
Register Renaming

- One method of coping with this is **register renaming**.
- Registers are allocated dynamically by the processor hardware, and they are associated with the values needed by the instructions at various points in time.
- When a new register value is created (i.e., an instruction has a register as a destination), a new register is created for that value.



Register Renaming

- Subsequent instructions that access that value as a source operand on that register must go through a renaming process:
 - The register references in those instructions must be revised to refer to the register containing the needed value.
 - Thus, the same original register reference in several different instructions may refer to different actual registers.

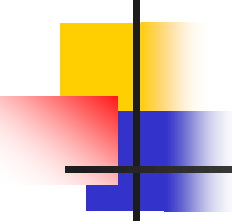


Register Renaming

- Consider again the code fragment:

```
I1:    R3b ← R3a op R5a
I2:    R4b ← R3b + 1
I3:    R3c ← R5a + 1
I4:    R7b ← R3c op R4b
```

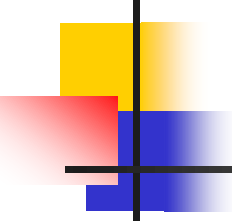
- The register reference without the subscript refers to the logical register reference found in the instruction.
- The register reference with the subscript refers to a hardware register allocated to hold this new value.



Register Renaming

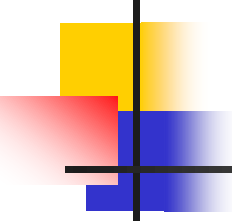
I1: $R3_b \leftarrow R3_a \text{ op } R5_a$
I2: $R4_b \leftarrow R3_b + 1$
I3: $R3_c \leftarrow R5_a + 1$
I4: $R7_b \leftarrow R3_c \text{ op } R4_b$

- When a new allocation is made for a particular logical register, subsequent instruction references to that logical register as a source operand are made to refer to the most recently allocated hardware register.
- In this example, the creation of register R3c in instruction I3 avoids the antidependency on the second instruction and the output dependency on the first instruction, and it does not interfere with the correct value being accessed by I4.
- The result is that I3 can be issued immediately; without renaming R3, I3 cannot be issued until the first instruction is complete and the second instruction is issued.



Machine Parallelism: Performance Gains

- We have looked at three hardware techniques that can be used in a superscalar processor to enhance performance:
 - Duplication of resources
 - Out-of-order issue
 - Register renaming



Machine Parallelism: Performance Gains

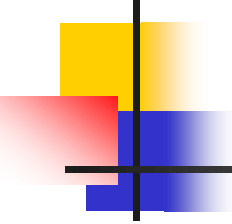
Analysis of Performance Gain (simulation):

Limited by all
dependencies

- **Without register renaming:**
 - Marginal improvement when duplicating functional units (memory access, ALU)
 - Marginal improvement with increasing instruction window size (for out-of-order issue).

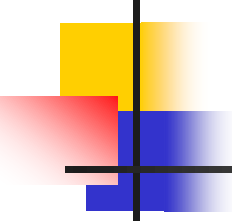
Limited only
by true data
dependencies

- **With register renaming:**
 - Dramatic improvements due to both.



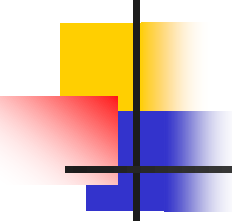
Machine Parallelism: Performance Gains

- It is not worthwhile to add functional units without register renaming.
 - Register renaming eliminates antidependencies and output dependencies.
- A significant gain is achievable by using an instruction window larger than 8 words.
 - If the window is too small, data dependencies will prevent effective utilization of the extra functional units; the processor must be able to look quite far ahead to find independent instructions to utilize the hardware more fully.



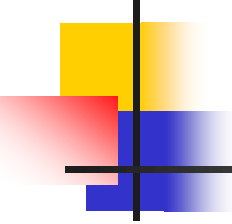
Branch Prediction

- Any high-performance pipelined machine must address the issue of dealing with branches.
- For example, the Intel 80486 fetches both the next sequential instruction after a branch and speculatively fetching the branch target instruction.
 - However, because there are two pipeline stages between prefetch and execution, this strategy incurs a two-cycle delay when the branch gets taken.



Branch Prediction

- With the advent of RISC machines, the delayed branch strategy was explored. This allows the processor to calculate the result of conditional branch instructions before any unusable instructions have been prefetched.
 - The processor always executes the single instruction immediately after the branch.
- This is less appealing with superscalar machines, as multiple instructions must execute in the delay slot, raising several problems relating to instruction dependencies.

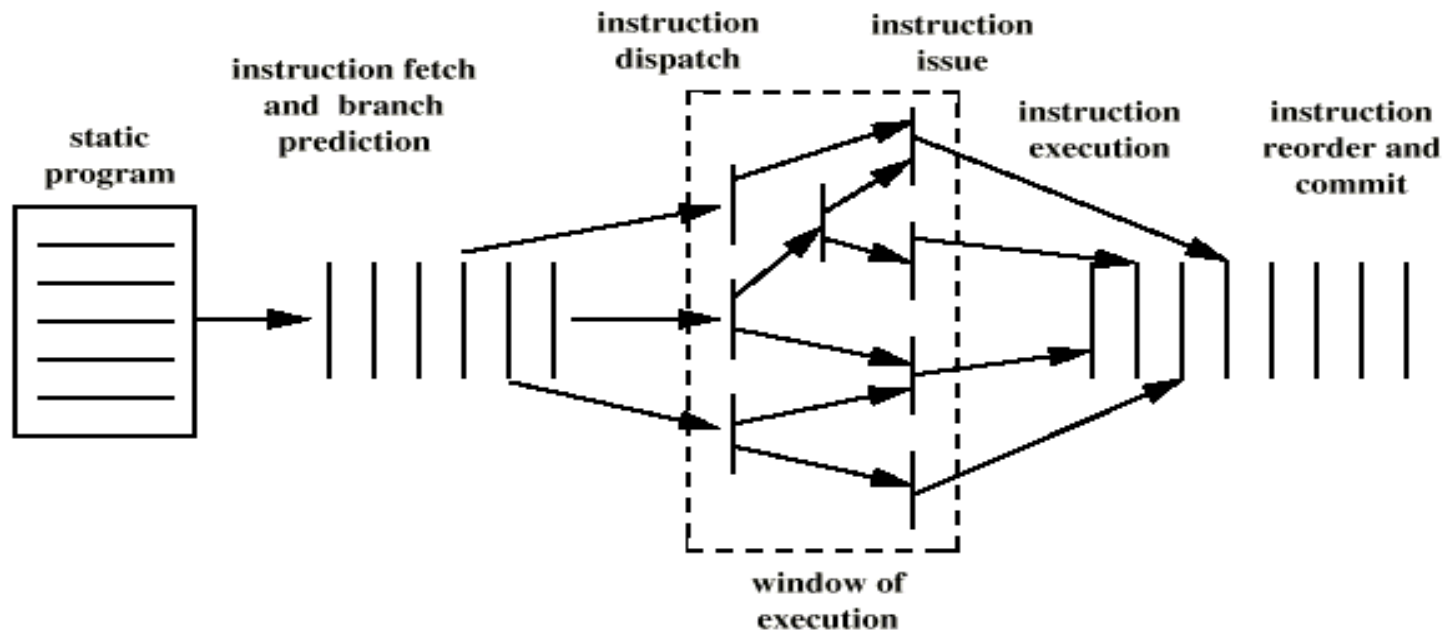


Branch Prediction

- Thus, some superscalar machines have turned to pre-RISC techniques of branch prediction.
 - The PowerPC 601 uses simple static branch prediction.
 - More sophisticated processors, such as the PowerPC 620 and the Pentium II, use dynamic branch prediction based on branch history analysis.

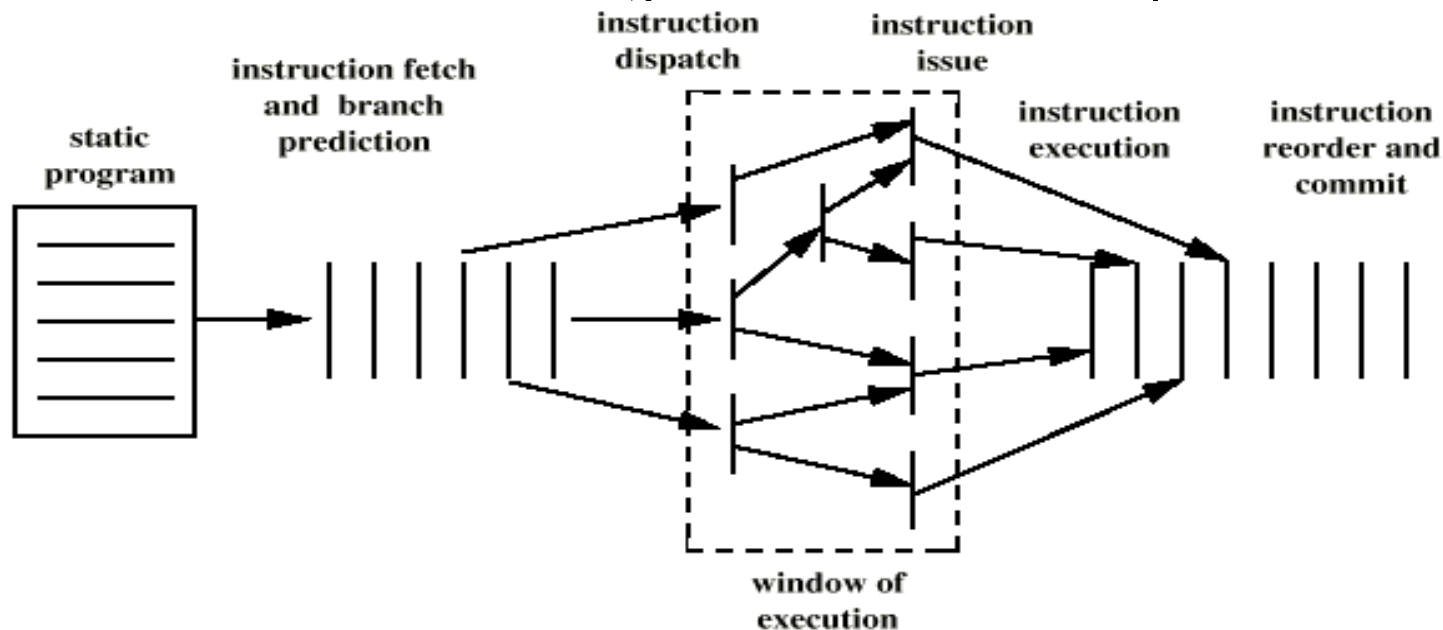
Superscalar Execution

- The program to be executed consists of a linear sequence of instructions (static program written by programmer or generated by compiler).
- The instruction fetch process, which includes branch prediction, is used to form a dynamic stream of instructions.



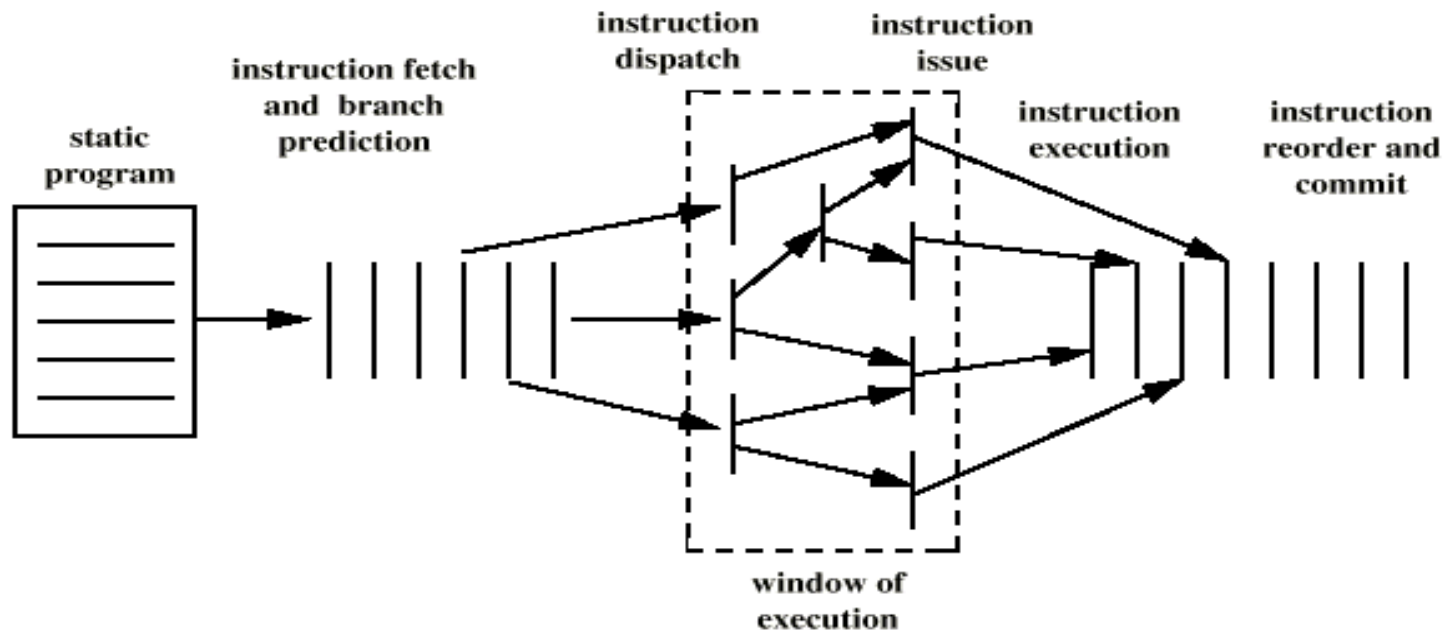
Superscalar Execution

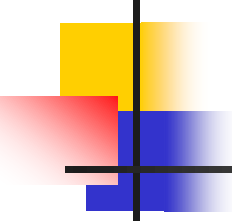
- This stream is examined for dependencies, and the processor may remove artificial dependencies.
- The processor then dispatches the instructions into a window of execution.
- In this window, instructions no longer form a sequential stream, but are structured according to their true data dependencies.



Superscalar Execution

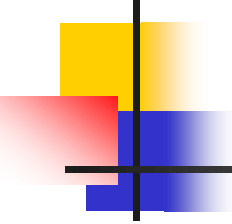
- The processor performs the execution stage of each instruction in an order determined by the true data dependencies and hardware resource availability.
- Finally, instructions are conceptually put back into sequential order and their results are recorded.





Superscalar Execution

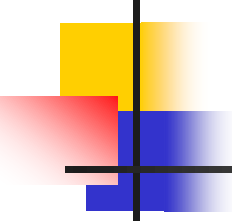
- This final step is referred to as **committing** or **retiring** the instruction.
- It is needed for the following reason:
 - Because of the use of parallel, multiple pipelines, instructions may complete in an order different from the original static program.
 - Further, the use of branch prediction and speculative execution means that some instructions may complete execution and then must be abandoned because the branch they represent is not taken.
 - Therefore, permanent storage and program-visible registers cannot be updated immediately when instructions complete execution.
 - Results must be held in some sort of temporary storage that is usable by dependent instructions and then made permanent when it is determined that the sequential model would have executed the instruction.



Superscalar Implementation

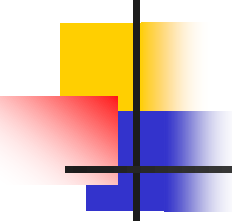
We can make some general comments about the processor hardware required for the superscalar approach:

- Instruction fetch strategies that simultaneously fetch multiple instructions,
 - Ability to predict (and fetch beyond) the outcome of conditional branch instructions.
 - This requires the use of multiple pipeline fetch and decode stages, and branch prediction logic.



Superscalar Implementation

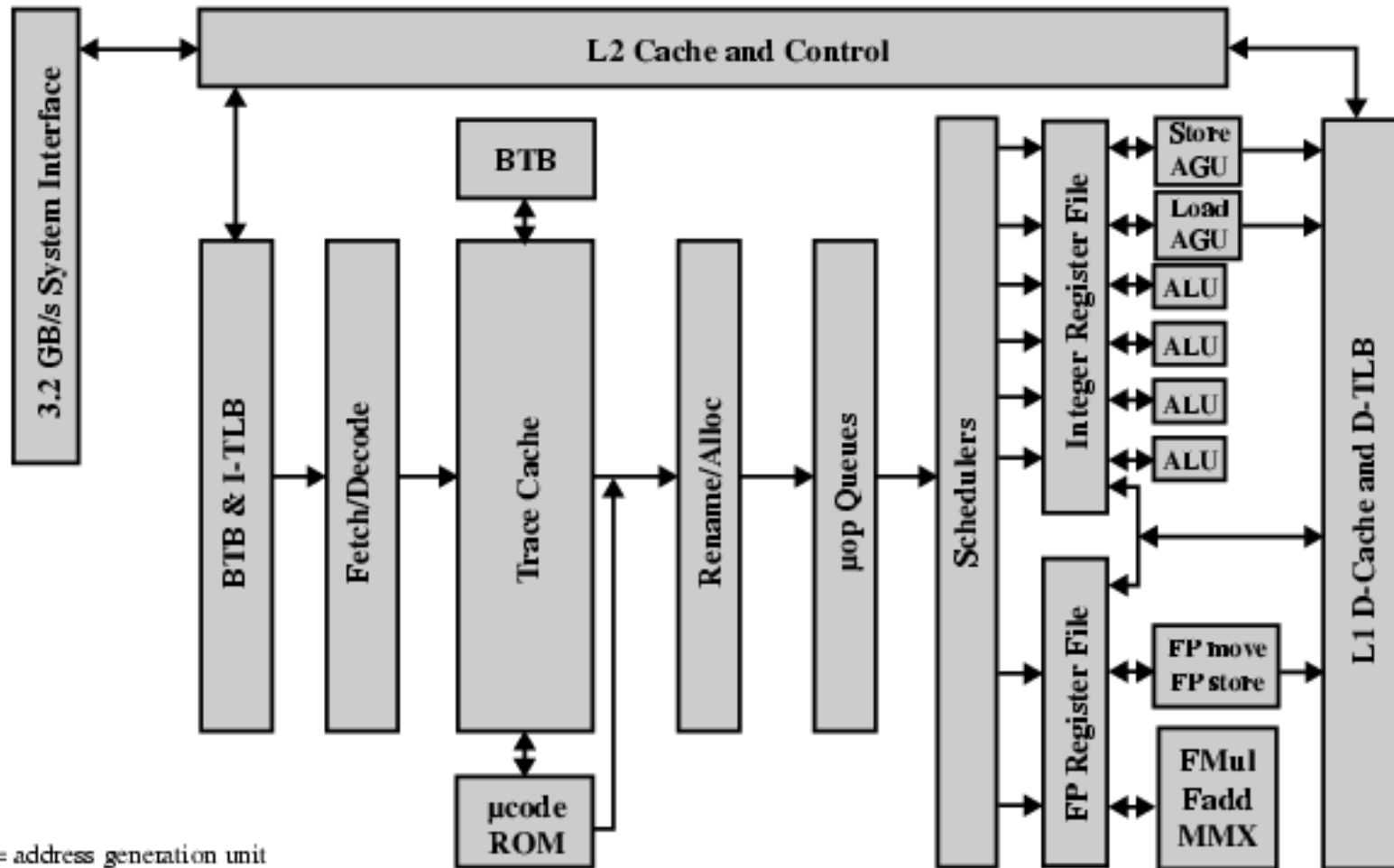
- Logic for determining true data dependencies involving register values.
 - Logic for register renaming.
- Mechanisms for issuing multiple instructions in parallel.
- Resources for parallel execution of multiple instructions
 - multiple pipelined functional units
 - memory hierarchies capable of simultaneously servicing multiple memory references.
- Mechanisms for committing the process state in correct order.



13.3 Pentium 4

- Although the concept of superscalar design is usually associated with the RISC architecture, superscalar principles can be applied to a CISC machine.
- The 80486 was a straightforward traditional CISC machine, with no superscalar elements.
- The original Pentium had modest superscalar elements:
 - Two separate integer execution units.
- Pentium Pro: full-blown superscalar design.
- Subsequent Pentium models have refined and enhanced the superscalar design.

Pentium 4

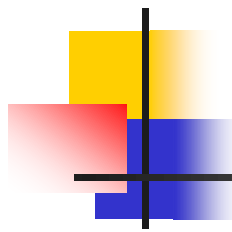


AGU = address generation unit

BTB = branch target buffer

D-TLB = data translation lookaside buffer

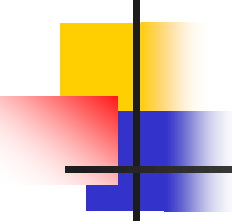
I-TLB = instruction translation lookaside buffer



Pentium 4

The operation of the Pentium II can be summarized as:

- The processor fetches instructions from memory in the order of the static program.
- Each instruction is translated into one or more fixed-length RISC instructions, known as micro-operations, or micro-ops.
- The processor executes the micro-ops on a superscalar pipeline organization, so that the micro-ops may be executed out of order.
- The processor commits the results of each micro-op execution to the processor's register set in the order of the original program flow.



Pentium 4

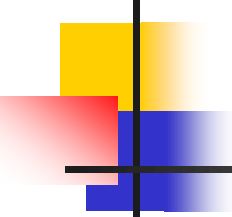
- In effect, the Pentium 4 organization consists of an outer CISC shell with an inner RISC core.
- The inner RISC micro-ops pass through a pipeline with at least 20 stages (compared to 5 on 486 and Pentium, 11 on Pentium II).



TC Next IP = trace cache next instruction pointer
TC Fetch = trace cache fetch
Alloc = allocate

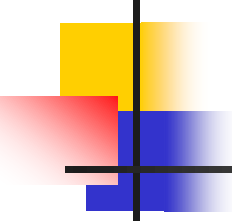
Rename = register renaming
Que = micro-op queuing
Sch = micro-op scheduling
Disp = Dispatch

RF = register file
Ex = execute
Flgs = flags
Br Ck = branch check



Pentium 4

- In some cases, the micro-op requires multiple execution stages, resulting in an even longer pipeline.
- ROB:
 - A circular buffer that can hold up to 126 micro-ops, and also contains 128 hardware registers.
 - Micro-ops enter the ROB in order.
 - Micro-ops are then dispatched from the ROB to the dispatch/execute unit out of order. The criterion for dispatch is that the appropriate execution unit and all necessary data items required for the micro-op are available.
 - The micro-ops are retired from the ROB in order.



13.4 PowerPC

- The PowerPC is a direct descendent of the IBM 801, the RT PC and the RS/6000.
- All of these are RISC machines, but the first to exhibit superscalar features was the RS/6000.
- Subsequent PowerPC models carry the superscalar concept further.
- The PowerPC 601:
 - Three independent pipelined execution units: integer, floating-point, and branch processing) superscalar of degree three).

PowerPC

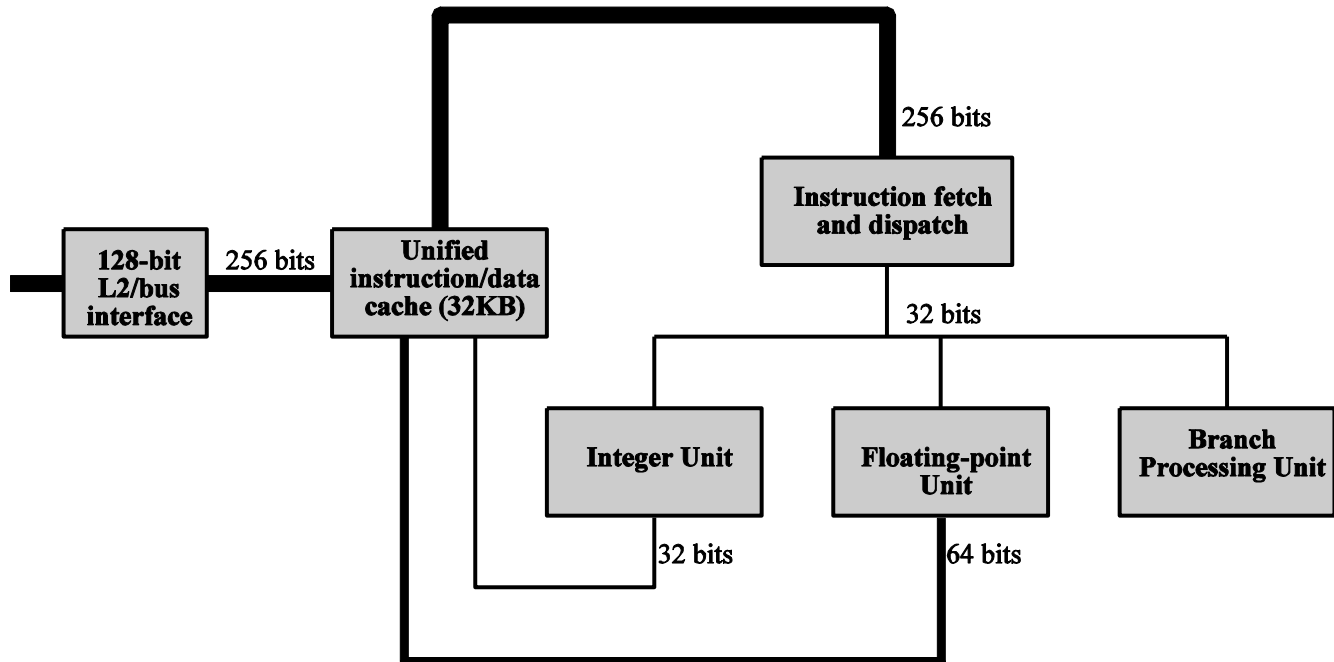
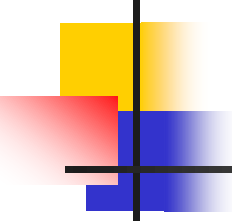


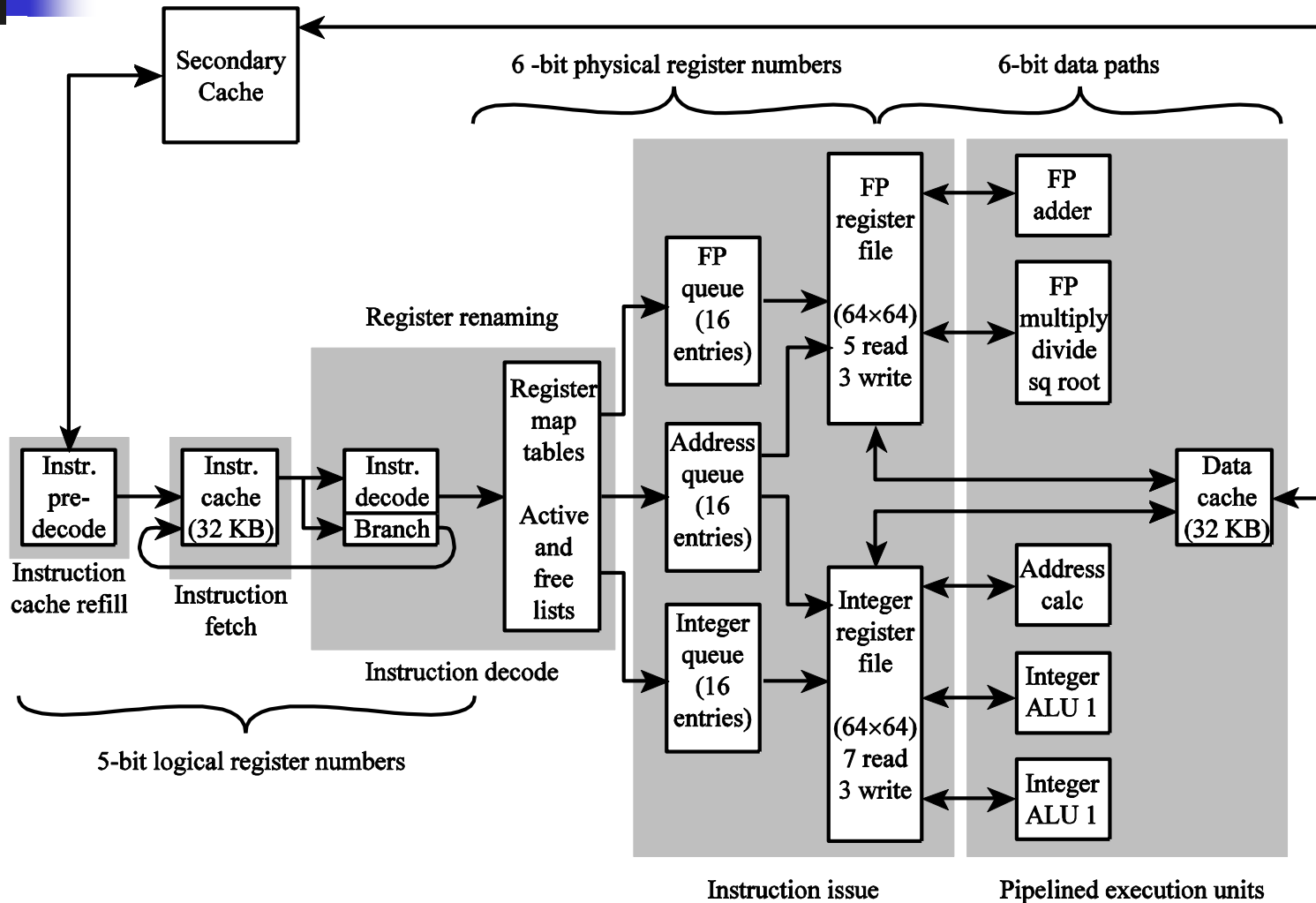
Figure 13.10 PowerPC 601 Block Diagram

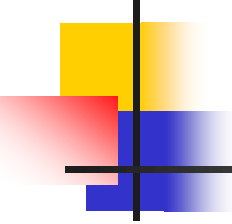


13.5 MIPS R10000

- The MIPS R10000, which has evolved from the MIPS R4000, is a clean, straightforward implementation of superscalar design principles.

MIPS R10000





MIPS R10000

- Predecode: classifies incoming instructions to simplify subsequent decode.
- Register renaming: removes false data dependencies.
- Three instruction queues: floating point, integer, load/save operations.
- Five execution units: address calculator, two integer ALU's, floating-point adder, floating-point unit for multiply, divide and square root.

UltraSparc-II

- A superscalar machine derived from the SPARC processor.

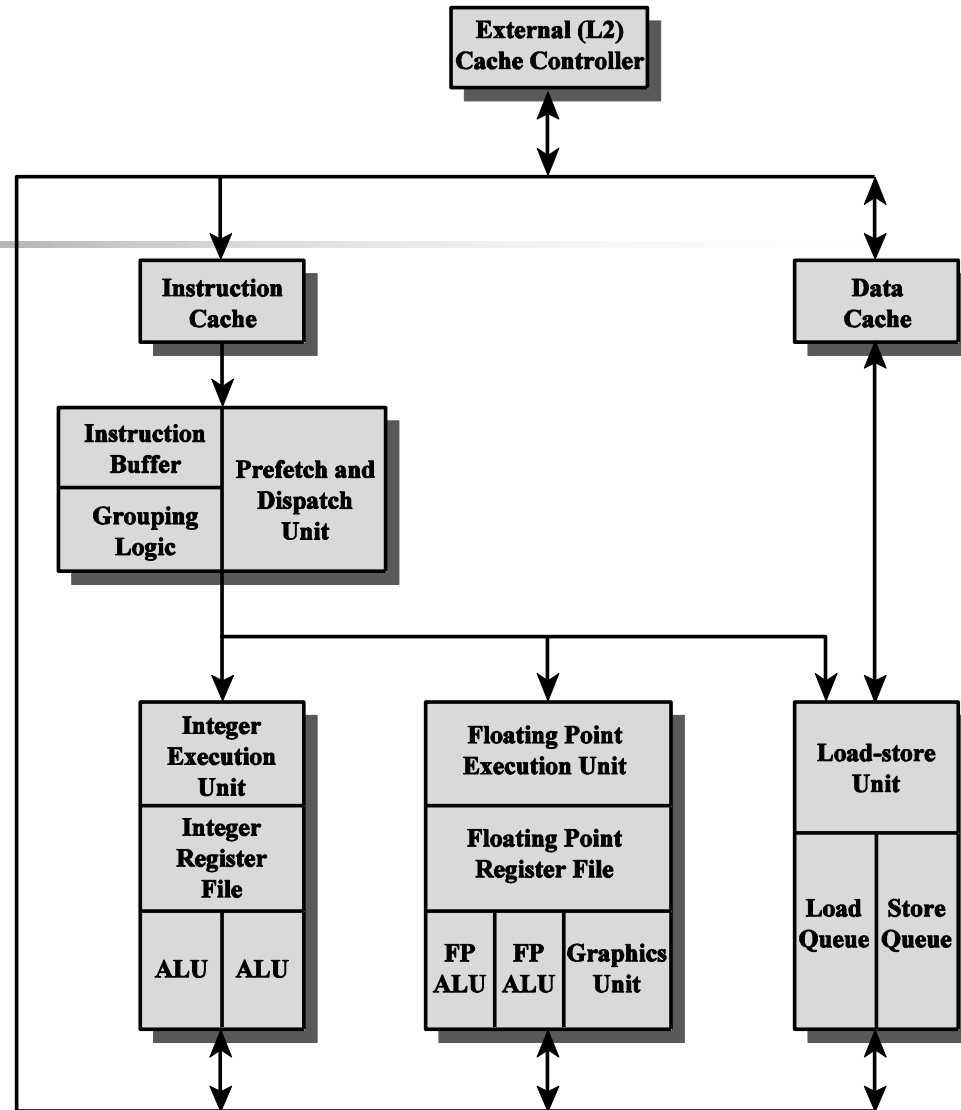


Figure 13.16 UltraSPARC-IIi Block Diagram

UltraSparc-II

- Prefetch and dispatch unit:
 - Fetches into instruction buffer
 - Responsible for branch prediction
- Grouping logic: organizes incoming instructions in to groups of up to four simultaneous instructions for simultaneous dispatch.
 - Each group may have two integer and two floating point/graphics instructions.

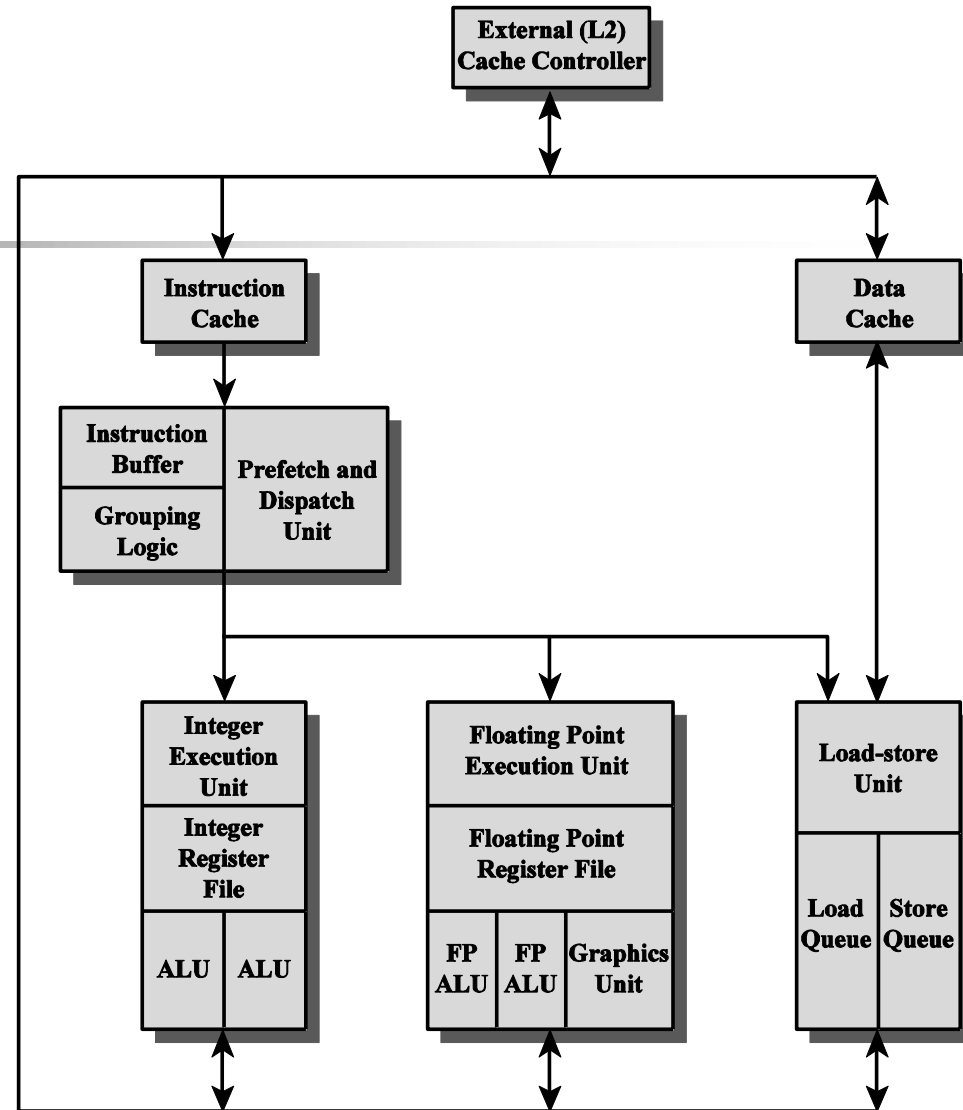


Figure 13.16 UltraSPARC-IIi Block Diagram

UltraSparc-II

- **Integer Execution Unit:** two integer ALU's that operate independently.
- **Floating-Point Unit:** two floating-point ALU's and a graphics unit: two FP instructions or one FP/one graphics instruction in parallel.
- **Graphics Unit:** supports visual instruction set (VIS) extension to the SPARC instruction set (similar to the MMX instruction set on the Pentium).
- **Load/Store Unit:** generates virtual address of all memory accesses.

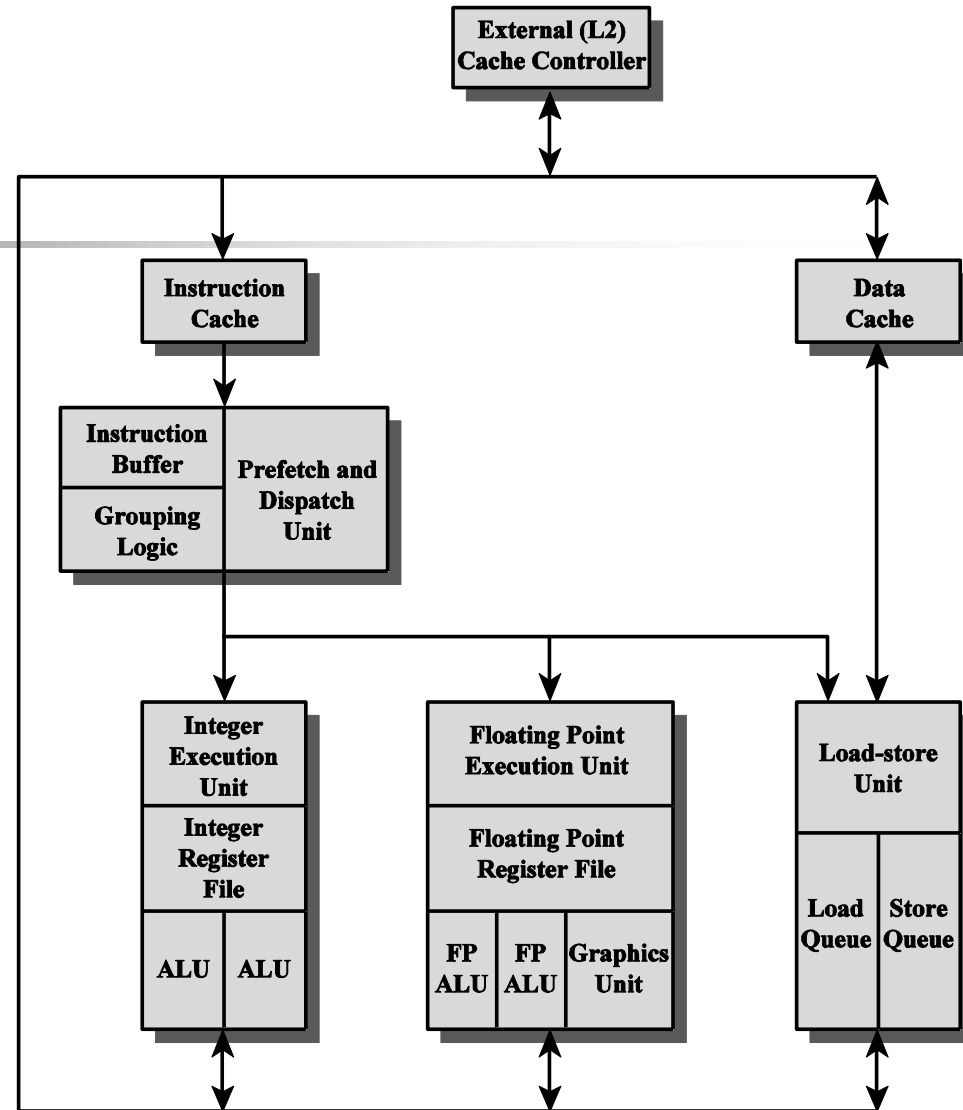


Figure 13.16 UltraSPARC-IIi Block Diagram